Instalacja i minimalny przykład komputerowej ewolucji gwiazdy w programie MESA

Linux, macOS, Windows (WSL i natywnie)

Andrzej Odrzywołek
andrzej.odrzywolek@uj.edu.pl

13 października 2026

Przykład to absolutnie minimalna konfiguracja MESA (26.04.1) dla ewolucji gwiazdy o masie $1\,M_\odot$ od jednorodnego obłoku H/He (pre-MS) do stygnącego białego karła, ok. 15 000 kroków. Obliczenia trwają od poniżej 1 godziny (szybki komputer) do kilku godzin.

Od czego zacząć:

- Pracownia komputerowa Fort OA (MESA już zainstalowana): rozdział 2.
- Własny komputer z Linuksem: rozdział 3. Działa także w maszynie wirtualnej, np. VMware Workstation pod Windows.
- Mac z procesorem Apple Silicon (M1–M4): rozdział 4.
- Windows: **zalecane** WSL (pełna funkcjonalność, z wykresami), rozdział 5. Bez WSL: wersja natywna (eksperymentalna, na razie bez wykresów), rozdział 6.

Spis treści

1 Wymagania **2**

2 Pracownia komputerowa Fort OA **2**

3 Linux (Ubuntu) – instalacja na własnym komputerze **3**

4 macOS (Apple Silicon) **4**

5 Windows – WSL (zalecane) **5**

6 Windows – natywnie (PowerShell 7, bez WSL) **5**

7 Uruchomienie przykładu (wszystkie systemy) **6**
7.1 Komendy programu MESA 6
7.2 Pełna ewolucja 6
7.3 Szybki test: 50 kroków od ciągu głównego 6

8 Pamięć i równanie stanu: `use_FreeEOS` **7**

9 Animacje **8**

10 Tabela benchmarków **9**

A Windows natywnie: szczegóły i rozwiązywanie problemów **10**
A.1 Testy modułów (opcjonalnie) 10
A.2 Parametry `mesa_env.ps1` 10
A.3 Co zmieniono względem instalacji na Linuksie 10
A.4 Rozwiązywanie problemów 10
A.5 Zawartość katalogu `windows` 11

1 Wymagania

- Dysk: pełna instalacja MESA to ok. 21 GB, MESA SDK ok. 0.7 GB.
- Pamięć: 8 GB RAM to minimalne oficjalne wymagania. Jeżeli w pliku `inlist` ustawimy `use_FreeEOS = .false.` (od 12 października 2026 ustawienie domyślne) obliczenia zajmują najwyżej ok. 2.7 GiB (rozdział 8). Wcześniej sam program zajmował do 7.8 GiB i na komputerach pracownianych z 7.6 GiB obliczenia się zawieszały.
- Ścieżki do katalogów z MESA **nie mogą zawierać spacji**.
- MESA SDK w wersji kompatybilnej z użytą wersją MESA (Linux, WSL: TODO; macOS: 26.6.1 z MESA 26.04.1).

2 Pracownia komputerowa Fort OA

MESA wersja TODO jest zainstalowana systemowo w `/opt/MESA`. Wystarczającą ilość pamięci powyżej 8 GB mają w tej chwili komputery: Cep, Aqr, Eri, Sgr, Hya.

TODO: *sprawdzić, czy po zmianie z 12 października 2026 (rozdział 8) przykład działa na wszystkich komputerach pracowni.*

1. W konsoli Linuxowej ustawiamy z linii komend zmienne środowiskowe dla MESA SDK i programu MESA:

   ```
   export MESASDK_ROOT=/opt/MESA/mesasdk
   source $MESASDK_ROOT/bin/mesasdk_init.sh
   export MESA_DIR=/opt/MESA/mesa-24.08.1
   ```

2. Aby uniknąć problemów z zapisem plików cache (eos – równanie stanu, kap – nieprzezroczystość, rates – reakcje jądrowe, ionization – jonizacja), tworzymy własny katalog (tylko za pierwszym razem) i ustawiamy zmienną `MESA_CACHES_DIR`:

   ```
   mkdir -p $HOME/.mesa
   export MESA_CACHES_DIR=$HOME/.mesa
   ```

3. Pracujemy w katalogu `/data/rok4` (zamień `rok4` na swój rocznik). Testy pokazują, że symulacja działa tam ok. 3 razy szybciej niż w katalogu `$HOME`, ale wyniki nie są dostępne na innych komputerach. Praca w katalogu domowym jest możliwa, program po prostu działa wolniej, ale za to wyniki są od razu dostępne na wszystkich stanowiskach pracowni komputerowej.

   ```
   cd /data/rok4
   mkdir MESA2027
   cd MESA2027
   git clone https://github.com/VA00/MESA-minimal-test-1MSun-star.git
   cd MESA-minimal-test-1MSun-star
   ```

4. Kompilacja i uruchomienie: `./mk`, `./rn` (rozdział 7).

Znany problem. Na komputerach pracowni obliczenia zawieszały się przy kroku 10 703 (okolice błysku helowego). TODO: Prawdopodobna przyczyna i rozwiązanie: rozdział 8.

Zadanie. Proszę powtórzyć testowy run w katalogach `/data/rok4` oraz `$HOME` i podać konkretne wartości czasu obliczeń.

3 Linux (Ubuntu) – instalacja na własnym komputerze

Sprawdzone 12 października 2026 na Ubuntu 24.04 (MESA SDK 24.7.1, MESA 24.08.1).

1. Pobieramy MESA SDK (24.7.1, ok. 209 MB, ok. 8 min) i MESA (24.08.1, ok. 2 GB, ok. 18 min). Serwer SDK odrzuca pobieranie z konsoli, dlatego `wget` „udaje” przeglądarkę Firefox:

   ```
   mkdir ~/MESA
   cd ~/MESA
   wget -U firefox \
   http://user.astro.wisc.edu/~townsend/resource/download/mesasdk/mesasdk-x86_64-linux-24.7.1.tar.gz
   wget https://zenodo.org/records/13353788/files/mesa-24.08.1.zip
   ```

2. Rozpakowujemy SDK i ustawiamy zmienne (bash):

   ```
   tar xvfz mesasdk-x86_64-linux-24.7.1.tar.gz -C ~/MESA/
   export MESASDK_ROOT=~/MESA/mesasdk
   source $MESASDK_ROOT/bin/mesasdk_init.sh
   ```

   Powinny pojawić się komunikaty:

   ```
   mesasdk_init.sh: checking architecture
   mesasdk_init.sh: regenerating headers
   mesasdk_init.sh: checking prerequisites
   ```

3. Na ogół brakuje kilku programów, o czym informuje komunikat typu:

   ```
   mesasdk_init.sh: missing prerequisites:
     C shell Z library X11 library
   ```

   Doinstalowujemy je i powtarzamy krok 2 (`source ...`). Instalujemy też `git` i `unzip` (kroki 4 i 7), których może brakować w świeżej instalacji Ubuntu:

   ```
   sudo apt install tcsh zlib1g-dev libx11-dev git unzip
   ```

4. Rozpakowujemy MESA i ustawiamy kolejne zmienne (liczba wątków według liczby rdzeni, rozdział 10):

   ```
   cd ~/MESA
   unzip mesa-24.08.1.zip
   export MESA_DIR=~/MESA/mesa-24.08.1
   export OMP_NUM_THREADS=4
   ```

5. Opcjonalnie, żeby zmienne ustawiały się przy każdym uruchomieniu terminala, dopisujemy je do `~/.bashrc`:

   ```
   echo 'export MESASDK_ROOT=~/MESA/mesasdk' >> ~/.bashrc
   echo 'source $MESASDK_ROOT/bin/mesasdk_init.sh' >> ~/.bashrc
   echo 'export MESA_DIR=~/MESA/mesa-24.08.1' >> ~/.bashrc
   echo 'export OMP_NUM_THREADS=4' >> ~/.bashrc
   ```

6. Instalacja (kompilacja trwa kilkanaście minut):

   ```
   cd $MESA_DIR
   ./install
   ```

   Jeżeli się powiodła, pojawi się dużo gwiazdek i napis:

   ```
   ********************************************
   MESA installation was successful
   ```

```
***********************************************
```

7. Pobieramy przykład:

   ```
   cd ~/MESA
   git clone https://github.com/VA00/MESA-minimal-test-1MSun-star.git
   cd MESA-minimal-test-1MSun-star
   ```

   i uruchamiamy go poleceniami `./mk`, `./rn` (rozdział 7).

Instalacja na dysku zewnętrznym. Na systemach plików bez dowiązań symbolicznych (exFAT, FAT32) SDK i MESA nie działają. Można je zainstalować na dysku lokalnym, a potem skopiować na dysk USB z rozwinięciem dowiązań:

```
rsync -aL --info=progress2 ~/MESA/mesa-24.08.1/ /media/$USER/USB/mesa-24.08.1/
rsync -aL --info=progress2 ~/MESA/mesasdk/ /media/$USER/USB/mesasdk/
```

i ustawić `MESA_DIR` oraz `MESASDK_ROOT` na nowe położenie.

4 macOS (Apple Silicon)

Aktualizacja: 13 października 2026 (MESA SDK 26.6.1, MESA 26.04.1).

1. Pobieramy i instalujemy MESA SDK dla Apple Silicon, wersja 26.6.1 (czerwiec 2026), ze strony `http://user.astro.wisc.edu/~townsend/static.php?ref=mesasdk` plik `mesasdk-aarch64-macos-26.6.1.pkg`. Instalacja jest okienkowa, po kliknięciu na pobraną „paczkę". SDK trafia do `/Applications/mesasdk`. Starsza wersja 24.10.1 już nie działa.

2. Zmienne środowiskowe (domyślna powłoka zsh jest zgodna z bash):

   ```
   export MESASDK_ROOT=/Applications/mesasdk
   source $MESASDK_ROOT/bin/mesasdk_init.sh
   ```

3. Pobieramy MESA (plik zip z Zenodo, `https://zenodo.org/records/19722306`). Safari sam rozpakowuje archiwum. Cały katalog `mesa-26.04.1` najlepiej przenieść do `~/MESA`. To samo z terminala:

   ```
   mkdir ~/MESA
   cd ~/MESA
   curl -L -O https://zenodo.org/records/19722306/files/mesa-26.04.1.zip
   unzip mesa-26.04.1.zip
   ```

4. Kolejne zmienne:

   ```
   export MESA_DIR=$HOME/MESA/mesa-26.04.1
   export OMP_NUM_THREADS=8
   export PATH=$PATH:$MESA_DIR/scripts/shmesa
   ```

   Żeby ustawiały się automatycznie, dopisujemy powyższe linie (także te z kroku 2) do `~/.zshrc`.

5. Instalacja:

   ```
   cd $MESA_DIR
   ./install
   ```

6. Aby MESA cokolwiek wyświetlała (okna pgstar), trzeba zainstalować serwer X, np. XQuartz (`https://www.xquartz.org`).

7. Pobieramy przykład i uruchamiamy go jak w Linuksie:

   ```
   cd ~/MESA
   git clone https://github.com/VA00/MESA-minimal-test-1MSun-star.git
   ```

```
cd MESA-minimal-test-1MSun-star
```

Czas obliczeń: 135 min przy `OMP_NUM_THREADS=8` na procesorze M3 Max.

Zadanie. Nowoczesne procesory (Apple M3, M4, ale też np. Intel i9 285) mają rdzenie typu „performance" (szybkie, gorące) i „efficiency" (wolniejsze, chłodniejsze). W Linuksie można przypisać symulację do wybranych rdzeni poleceniem `taskset`. Jak zrobić to w macOS?

5 Windows – WSL (zalecane)

TODO: co to za brednie?

1. W PowerShell **uruchomionym jako administrator**, potem restart komputera:

   ```
   wsl --install
   ```

   W oknie Ubuntu, które otworzy się po restarcie, zakładamy użytkownika i hasło.

2. Pamięć dla WSL. W zwykłym PowerShell:

   ```
   notepad $env:USERPROFILE\.wslconfig
   ```

   wpisujemy (np. dla komputera z 16 GB RAM)

   ```
   [wsl2]
   memory=12GB
   ```

   zapisujemy plik i wykonujemy `wsl --shutdown`.

3. W oknie Ubuntu:

   ```
   sudo apt update
   sudo apt install tcsh zlib1g-dev libx11-dev binutils make perl unzip wget git
   ```

   i dalej według rozdziału 3, w katalogu `~/MESA` (**nie** w `/mnt/c`).

6 Windows – natywnie (PowerShell 7, bez WSL)

Wersja natywna nie wyświetla wykresów (pgstar). Wszystkie polecenia wpisujemy w PowerShell 7.

1. Instalujemy PowerShell 7, jeżeli ktoś jeszcze go nie ma:

   ```
   winget install --id Microsoft.PowerShell -e
   ```

   Dalej pracujemy w oknie „PowerShell 7". Uruchamiamy z menu lub klawisz Win+R i „pwsh".

2. Instalujemy MSYS2 i kompilator gfortran:

   ```
   winget install --id MSYS2.MSYS2 -e
   C:\msys64\usr\bin\pacman.exe -Syu --noconfirm
   C:\msys64\usr\bin\pacman.exe -Syu --noconfirm
   C:\msys64\usr\bin\pacman.exe -S --needed --noconfirm mingw-w64-ucrt-x86_64-gcc-fortran
   C:\msys64\usr\bin\pacman.exe -S --needed --noconfirm mingw-w64-ucrt-x86_64-make
   C:\msys64\usr\bin\pacman.exe -S --needed --noconfirm mingw-w64-ucrt-x86_64-hdf5
   C:\msys64\usr\bin\pacman.exe -S --needed --noconfirm mingw-w64-ucrt-x86_64-lapack
   ```

3. Instalujemy Python i Git:

   ```
   winget install --id Python.Python.3.14 -e
   winget install --id Git.Git -e
   ```

   Zamykamy PowerShell 7 i uruchamiamy go ponownie.

4. Pobranie i rozpakowanie MESA (komunikaty `Can't create ...` można zignorować):

```
mkdir C:\MESA
cd C:\MESA
curl.exe -L -O https://zenodo.org/records/19722306/files/mesa-26.04.1.zip
tar -xf mesa-26.04.1.zip
```

5. Pobranie przykładu:

   ```
   git clone https://github.com/VA00/MESA-minimal-test-1MSun-star.git
   ```

6. Ustawiamy zmienne środowiskowe PowerShell:

   ```
   . C:\MESA\MESA-minimal-test-1MSun-star\windows\mesa_env.ps1
   ```

7. Kompilacja MESA (kilka minut, na końcu napis `MESA installation was successful`):

   ```
   C:\MESA\MESA-minimal-test-1MSun-star\windows\install_mesa.ps1
   ```

8. Kompilacja i uruchomienie przykładu:

   ```
   cd C:\MESA\MESA-minimal-test-1MSun-star
   ./mk
   ./rn
   ```

Dalej jak w rozdziale 7.

7 Uruchomienie przykładu (wszystkie systemy)

7.1 Komendy programu MESA

kompilacja	`./mk`
start od początku	`./rn`
restart od ostatniego zdjęcia	`./re`
restart od zdjęcia `x500`	`./re x500`
sprzątanie kompilacji	`./clean`

Obliczenia przerywamy kombinacją `Ctrl+C`. Pliki restartu są w katalogu `photos`.

7.2 Pełna ewolucja

Po `./rn` po kilku minutach pojawiają się okna z wizualizacją ewolucji gwiazdy. Typowy koniec obliczeń:

```
save photos/x986 for model 14986
termination code: log_L_lower_limit

                runtime (minutes), retries, steps      ...          491     14986
```

Czas obliczeń zależy od komputera, patrz rozdział 10. Wyniki trafiają do katalogów `LOGS` (`history.data`, profile), `photos` (zdjęcia do restartów) i `png` (wykresy).

W trakcie ewolucji możemy edytować plik `inlist.pgstar`, w szczególności dodawać i zmieniać różne tryby wizualizacji wnętrza gwiazdy.

7.3 Szybki test: 50 kroków od ciągu głównego

Po kompilacji (`./mk`):

```
mkdir ../zams_test
cp star inlist inlist.pgstar history_columns.list rn ../zams_test/
cd ../zams_test
```

W Windows natywnie:

```
mkdir ../zams_test
Copy-Item star.exe, inlist, inlist.pgstar, history_columns.list, rn.ps1 ../zams_test
cd ../zams_test
```

W pliku `inlist` (edytor `nano inlist` lub `notepad inlist`) zmieniamy w sekcji `&star_job`

```
  create_pre_main_sequence_model = .false.
```

a w sekcji `&controls` dopisujemy linijkę

```
  max_model_number = 50
```

Uruchamiamy `./rn`. Poprawny wynik (0 powtórzeń, 50 kroków):

```
termination code: max_model_number
                runtime (minutes), retries, steps        0.10          0         50
```

Wynik sprawdzony w Windows natywnie i w Linuksie (także po zmianie z 12 października 2026), w macOS jeszcze nie.

Znane problemy.

- Błąd `Cannot match namelist object name read_extra_pgstar_inlist1`: w pliku `inlist` trzeba zmienić nazwy (w repozytorium już poprawione):

  ```
  read_extra_pgstar_inlist1   ->  read_extra_pgstar_inlist(1)
  extra_pgstar_inlist1_name   ->  extra_pgstar_inlist_name(1)
  ```

- Błąd związany z `adipls` może wynikać ze zmiany nazewnictwa nowych wersji MESA: nie ma literki „r”, tylko sam numer wersji.

8 Pamięć i równanie stanu: `use_FreeEOS`

Problem. Na komputerach pracowni (8 GB RAM, system widzi 7.6 GiB) obliczenia zawieszały się przy kroku 10 703, w okolicy błysku helowego.

Pomiar zużycia pamięci. Aby zmierzyć zużycie pamięci, używamy polecenia

```
grep VmRSS /proc/$(pgrep -n star)/status
```

Wynik jest na rysunku 1 (niebieska linia). Pamięć rośnie skokami do 7.2 GiB już przed ciągiem głównym (ok. 600. model), potem do 7.5 GiB przy zapłonie helu w jądrze i do 7.8 GiB przy późnym błysku w warstwie helowej. To więcej, niż ma komputer na pracowni, nawet bez systemu. Do kroku 10 703 obliczenia szły najprawdopodobniej dzięki pamięci wymiany (swap).

Liczba wątków nie ma znaczenia (4 i 24 wątki: ta sama pamięć w tych samych modelach), wykresy pgstar zajmują ok. 0.1 GiB. Dlatego wyłączanie wykresów czy zmniejszanie liczby wątków nie pomaga.

Przyczyna. Które pliki program czyta w chwili skoku pamięci, pokazuje

```
strace -f -e trace=openat -p $(pgrep -n star)
```

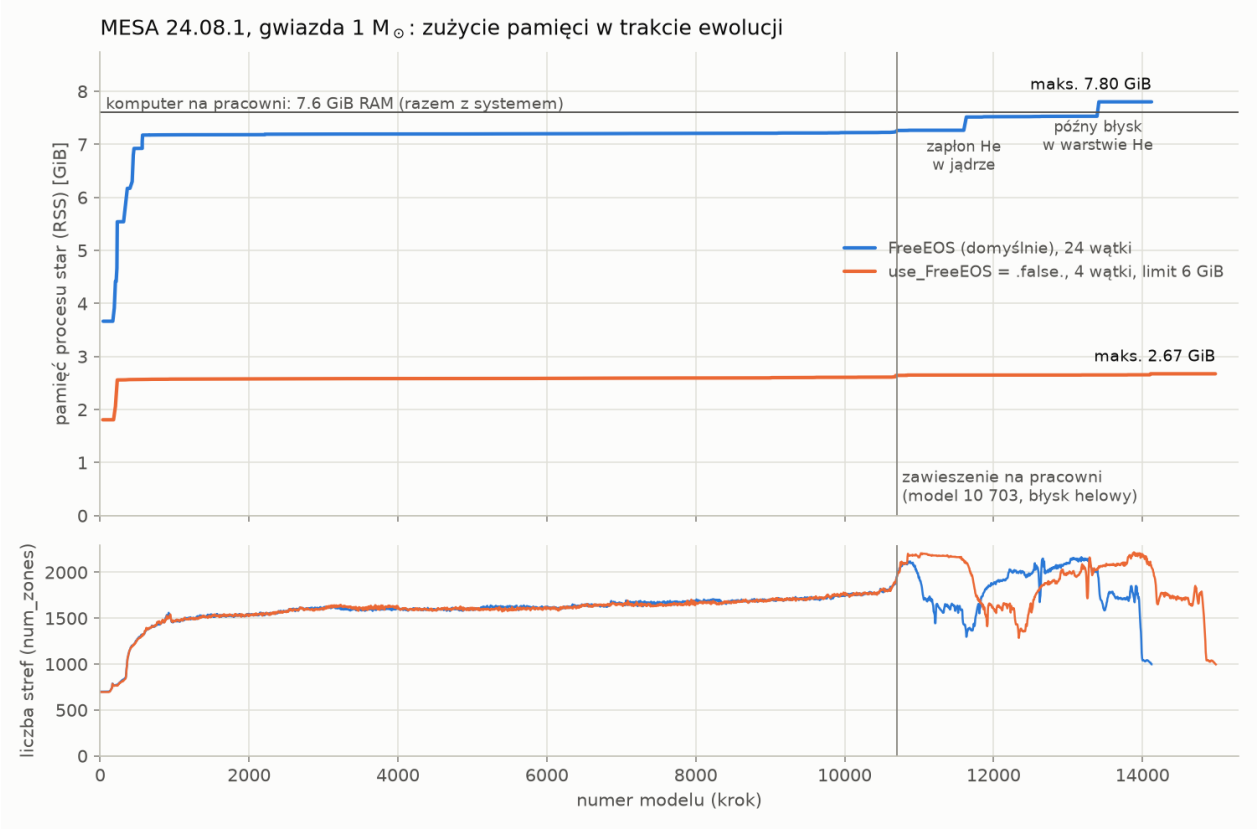
Każdy skok to wczytanie kolejnych tablic równania stanu FreeEOS, czyli plików z katalogu `$MESA_DIR/data/eosFreeEOS_data`. Jedna tablica zajmuje w pamięci 128 MB, a tablic jest 89 (różne zawartości wodoru X i metali Z), razem ok. 11 GB. MESA wczytuje je w miarę potrzeby i już nie zwalnia.

Rozwiązanie. W sekcji `&eos` pliku `inlist`:

```
use_FreeEOS = .false.
```

W obszarze, który obejmowało FreeEOS, MESA używa wtedy tablic OPAL/SCVH (a poza nimi HELM). Pamięć: najwyżej 2.7 GiB (pomarańczowa linia). Pełna ewolucja do białego karła przeszła z pamięcią ograniczoną do 6 GiB, bez swapu. Tak można na własnym komputerze z Linuksem sprawdzić, czy obliczenia zmieszczą się w mniejszej pamięci, bez maszyny wirtualnej:

```
systemd-run --user --scope -p MemoryMax=6G -p MemorySwapMax=0 ./rn
```



Rysunek 1: Pamięć programu `star` w trakcie ewolucji. Niebieski: z FreeEOS (do 12 października 2026), pomarańczowy: `use_FreeEOS = .false.`. Dolny panel: liczba stref modelu.

Uwaga. Tablice OPAL w MESA są tylko dla $Z \leq 0.04$. Przy większej metaliczności (`initial_z`) bez FreeEOS zostaje równanie stanu HELM, które zakłada pełną jonizację. Wtedy trzeba wrócić do `use_FreeEOS = .true.` (i komputera z 16 GB RAM).

Zadanie. Na komputerze z co najmniej 16 GB RAM uruchom przykład w osobnym katalogu z `use_FreeEOS = .true.` i porównaj z wynikiem domyślnym: diagram H-R, wiek na końcu ciągu głównego i na szczycie gałęzi czerwonych olbrzymów, masa jądra helowego, masa białego karła, liczba kroków i powtórzeń. Co się zmienia, a co nie? Dlaczego?

9 Animacje

Z plików w katalogu `png` można utworzyć film programem `ffmpeg` (w Linuksie jest częścią MESA SDK). Każdy typ wykresu ma własne pliki: `HR_*`, `Kippenhahn_*`, `abund_*`, `power_*`, `trho_profile_*`, `Summary_Burn_*`. Film robimy z jednego typu, np. diagramu H-R (wzorzec `png/*.png` skleiłby wszystkie typy jeden po drugim):

```
# film AVI
ffmpeg -f image2 -pattern_type glob -i 'png/HR_*.png' -c:v libx264 -preset slow \
  -crf 20 -pix_fmt yuv420p -aspect 16:9 -vf "scale=1920:-1" -r 25 HR.avi

# film MKV bezstratny
ffmpeg -f image2 -pattern_type glob -i 'png/HR_*.png' -c:v ffv1 -level 3 \
  -pix_fmt yuv444p -aspect 16:9 -vf "scale=1200:-1" -r 25 HR.mkv

# animacja GIF
ffmpeg -y -f image2 -thread_queue_size 256 -pattern_type glob -i 'png/HR_*.png' \
  -vf "scale=800:-1:flags=lanczos,palettegen" /tmp/palette.png
ffmpeg -y -f image2 -thread_queue_size 256 -pattern_type glob -i 'png/HR_*.png' \
  -i /tmp/palette.png \
  -lavfi "scale=1600:-1:flags=lanczos [x]; [x][1:v] paletteuse" HR.gif
```

Skrypt `./parallel_movie_encode.sh` tworzy równolegle (też programem `ffmpeg`) osobny film AVI dla każdego typu wykresu, który jest w katalogu `png`.

10 Tabela benchmarków

Czas obliczeń testowego modelu (w minutach) na różnych komputerach i przy różnej liczbie wątków `OMP_NUM_THREADS`. Czas jest podany w końcowych linijkach wydruku: `runtime (minutes), retries, steps`. Czasy w tabeli zmierzono przed 12 października 2026, z FreeEOS (303 powtórzenia, 14 122 kroki). Bez FreeEOS kroków jest ok. 6% więcej: Intel Core Ultra 9 285K, Ubuntu 24.04, 24 wątki: pomiar w toku (491 powtórzeń, 14 986 kroków).

Komputer	**wątki: czas [min]**
MacBook M3 Max	1: 747.52; 2: 391.84; 5: 181.67; 6: 160.08; 7: 144.87; 8: 134.33; 9: 123.51; 10: 113.25
VMware	4: 157.74
Pracownia OA Fort (Cep, Aqr, Eri, Sgr, Hya)	4: zawiesił się
Intel Core Ultra 9 285K, 128 GB RAM, Windows 11, WSL2	1: 411.39; 2: 220.32; 3: 155.52; 4: 120.57; 5: 99.37; 6: 85.45; 8: 70.12; 24: 42.39
Intel Core Ultra 9 285K, 128 GB RAM, Ubuntu 24.04	24: 55.19
Super Micro, 2× Intel Xeon Gold 6238R 2.2 GHz (2×26 rdzeni), 376 GB RAM, Ubuntu 22.04	1: 732.24; 2: 397.55; 3: 283.02; 4: 223.56; 5: 184.39; 6: 161.15; 7: 143.23; 8: 129.57; 9: 118.77; 10: 109.70; 24: 67.23
AMD Ryzen 7 7435HS 3.1 GHz, 16 GB RAM, Windows 11, WSL2	2: 393.79; 4: 198.33; 8: 152.86
MacBook Air M4, 16 GB RAM	2: nie skończył przez całe zajęcia
Intel i7-4770K 3.4 GHz, 16 GB RAM, Ubuntu	1: 783.99; 2: 420.83; 3: 308.45; 4: 246.79; 5: 237.08; 6: 227.10; 7: 218.53; 8: 210.82
AMD Ryzen 9 5900X, 32 GB RAM, Windows 11 natywnie (gfortran 16.1 MinGW)	12: 70.77
AMD Ryzen 9 5900X, 32 GB RAM, Windows 11 natywnie, MESA 26.04.1 bez FreeEOS (340 powtórzeń, 14 299 kroków)	12: 65.26

`OMP_NUM_THREADS` nie powinno być większe niż liczba wolnych rdzeni. Pomiar z 12 października 2026 (Intel Core Ultra 9 285K, 24 rdzenie, 24 wątki): gdy 4 rdzenie zajmował drugi rachunek MESA,

obliczenia szły 0.34 modelu na sekundę, a po jego zatrzymaniu 3.58 modelu na sekundę, czyli 10 razy szybciej.

A Windows natywnie: szczegóły i rozwiązywanie problemów

Sprawdzone 13 października 2026: Windows 11, AMD Ryzen 9 5900X, 32 GB RAM, MESA 26.04.1, gfortran 16.1.0, HDF5 2.1.1 i LAPACK 3.12.1 (MSYS2). Ograniczenia: brak wykresów pgstar (MESA zbudowana bez PGPLOT, `pgstar_flag` jest ignorowane), nie są budowane GYRE, ADIPLS, `binary` i `astero`, wyniki nie są identyczne bit w bit z Linuksem (różnice na poziomie zaokrągleń).

A.1 Testy modułów (opcjonalnie)

Po instalacji MESA, w oknie z ustawionymi zmiennymi (kilkanaście minut):

```
C:\MESA\MESA-minimal-test-1MSun-star\windows\test_mesa.ps1
```

Wyniki są porównywane z wzorcami z Linuksa (tolerancja 10^{-8}). Oczekiwane: `OK` wszędzie oprócz `num` (całkowanie sztywnych równań różniczkowych wzmacnia różnice zaokrągleń do 10^{-5}) i `net` (jedna linia, różnica $5 \cdot 10^{-7}$).

A.2 Parametry `mesa_env.ps1`

`-MesaDir` (inne położenie MESA niż katalog obok repozytorium), `-Threads` (liczba wątków, domyślnie liczba rdzeni), `-Toolchain` (MSYS2 w innym miejscu niż `C:\msys64`).

A.3 Co zmieniono względem instalacji na Linuksie

1. **System budowania.** Moduły są kompilowane oryginalnymi makefile'ami MESA, uruchamianymi natywnym `mingw32-make.exe` (receptury wykonuje `cmd.exe`), z plikiem `utils\makefile_header` w wersji dla Windows. Narzędzia z MESA SDK zastąpiono skryptami PowerShell (`makedepf90.ps1`, `cp_if_newer.ps1`), a bashowe `install` i `build_data_and_export` skryptem `install_mesa.ps1`. Biblioteka forum (z GYRE) jest preprocesowana programem `fypp`. LAPACK/BLAS i HDF5 pochodzą z MSYS2 (linkowane statycznie). GYRE jest zastąpione stubem.

2. **Plik C** `utils\private\utils_c_system.c` używa funkcji POSIX, których MinGW nie ma w tej postaci (`mkdir(path, mode)`, `realpath`, brak `O_BINARY`), oraz `rename()`, które w Windows nie nadpisuje istniejącego pliku. Łata jest ujęta w `#ifdef _WIN32`.

3. **Ustawienia regionalne.** libgfortran (MinGW/UCRT) przestawia proces na locale użytkownika (`Polish_Poland.1250`, przecinek dziesiętny) i w wątkach OpenMP liczby z kropką są czytane błędnie. `mesa_win_locale.c` przechwytuje `setlocale` (`-Wl,--wrap=__imp_setlocale`, libgfortran linkowana statycznie) i utrzymuje locale „C".

4. **Stos.** Windows daje głównemu wątkowi 2 MB stosu (Linux: 8 MB lub `ulimit -s unlimited`). `star.exe` jest linkowany z rezerwacją 1 GB (`-Wl,--stack,1073741824`), a wątkom OpenMP `mesa_env.ps1` ustawia `OMP_STACKSIZE=512M`.

5. **Drobiazgi.** W `chem\make\makefile_base` usunięto `-m %m.mod` (`cmd.exe` traktuje `%m` jak zmienną). Dowiązanie `pgstar.f90` zastępuje kopia `pgstar_stub.f90`. `star.exe` potrzebuje `libgomp-1.dll` i `libwinpthread-1.dll` z `C:\msys64\ucrt64\bin` (dodaje je do `PATH` `mesa_env.ps1`).

A.4 Rozwiązywanie problemów

Brak zmiennych środowiskowych Komunikaty `MESA_DIR is not set`, `mingw32-make` nie jest rozpoznawane albo okno „nie znaleziono libgomp-1.dll": w tym oknie nie wykonano

`. ...\windows\mesa_env.ps1` (z kropką na początku). MSYS2 w innym miejscu niż `C:\msys64`: parametr `-Toolchain D:/msys64/ucrt64`.

Skrypty są blokowane („running scripts is disabled on this system") Jednorazowo zezwalamy na skrypty:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

`pacman`: błędy pobierania pakietów (404, `failed retrieving file`) Nieaktualna baza pakietów: `C:\msys64\usr\bin\pacman.exe -Syu --noconfirm` i ponowna instalacja.

`tar: Error opening archive` przy pliku `.zip` Przed systemowym `tar` w `PATH` jest GNU tar (z Git lub MSYS2). Pełna ścieżka: `C:\Windows\System32\tar.exe -xf mesa-26.04.1.zip`.

`Python 3 not found`, otwiera się Microsoft Store Zainstaluj Pythona i otwórz nowe okno PowerShell. Alias Sklepu można wyłączyć w *Ustawienia → Aplikacje → Zaawansowane ustawienia aplikacji → Aliasy wykonywania aplikacji*.

Pierwsze uruchomienie jest bardzo wolne Tworzenie plików cache (jednorazowo). Rozpakowywanie i kompilację spowalnia też antywirus. Można (jako administrator) wyłączyć skanowanie katalogu `C:\MESA`.

A.5 Zawartość katalogu `windows`

Plik	Rola / odpowiednik na Linuksie
`mesa_env.ps1`	`export MESA_DIR=...`, `source mesasdk_init.sh`
`install_mesa.ps1`	`./install`
`test_mesa.ps1`	testy modułów (`build_and_test`)
`makefile_header`	`utils/makefile_header` dla Windows
`forum_makefile`	kompilacja biblioteki forum
`tools\makedepf90.ps1`	`makedepf90` z MESA SDK
`tools\cp_if_newer.ps1`	`utils/cp_if_newer`
`tools\mesa_win_locale.c`	poprawka locale (punkt 3)
`..\mk.ps1, rn.ps1, re.ps1, clean.ps1`	`./mk`, `./rn`, `./re`, `./clean`